LIVE DEMONSTRATION QUICK GUIDE

Linux Debugging System with Memory Leak Detection and Reporting Tool

Guided by: Dr. K. Anusha | **Hackathon:** 17 October 2026

2520030034 P. Prathik	**2520030188** U. Praneeth	**2520030573** M. Harshith

1. Start the project

Open Ubuntu and enter these commands one at a time:

```
cd ~/hackathon/Linux-Debugging-System
make
./build/linux_debugging_system
```

Important: Type a menu number only when the program says *Choose an option (1-5):*. After one test finishes, the program returns to the Ubuntu prompt. Run the program again for the next test.

2. Show the five menu options

Option	What to show	Expected result
1 - Leak	Allocate 100 bytes and do not release the block.	**1 block / 100 bytes**; MEMORY LEAK DETECTED.
2 - Clean	Allocate 100 bytes and release the same block.	**0 blocks / 0 bytes**; no tracked leaks detected.
3 - Partial	Allocate 100, 200, 300 bytes; release 100 and 300.	**1 block / 200 bytes** remains.
4 - Release safety	Show the pointer-clearing repeated-release demonstration.	0 outstanding blocks; second call is a safe no-op because pointer is NULL.
5 - Help	Display supported modes and usage.	Help / usage information appears.

3. Show the report file

Run option 1 immediately before viewing the leak report. Each test overwrites the default report file.

```
cat reports/memory_report.txt
```

4. Short explanation to say to Madam

> *“Madam, our tool tracks memory operations made through our tracking functions. In this test, we allocated 100 bytes and intentionally did not release them. The tool reported one outstanding block of 100 bytes, showed where it was allocated, and saved the report. In the clean test, the allocated block was released and the report showed zero outstanding blocks. We also tested partial release and safe repeated-release handling.”*

Scope note: The prototype only tracks allocations made through TRACK_MALLOC / TRACK_CALLOC and releases routed through TRACK_FREE. The remaining blocks are reported as potential leaks.